arXiv:2305.13304v1 [cs.CL] 22 May 2023

RECURRENTGPT: Interactive Generation of (Arbitrarily) Long Text

Wangchunshu Zhou* **Yuchen Eleanor Jiang*** **Peng Cui** **Tiannan Wang**
Zhenxin Xiao **Yifan Hou** **Ryan Cotterell** **Mrinmaya Sachan**
ETH Zürich
{wangchunshu.zhou, yuchen.jiang, peng.cui}@inf.ethz.ch
hugothebestwang@gmail.com, alanshawzju@gmail.com
{yifan.hou, ryan.cotterell, mrinmaya.sachan}@inf.ethz.ch

Abstract

The fixed-size context of Transformer makes GPT models incapable of generating arbitrarily long text. In this paper, we introduce RECURRENTGPT, a language-based simulacrum of the recurrence mechanism in RNNs. RECURRENTGPT is built upon a large language model (LLM) such as ChatGPT and uses natural language to simulate the Long Short-Term Memory mechanism in an LSTM. At each timestep, RECURRENTGPT generates a paragraph of text and updates its language-based long-short term memory stored on the hard drive and the prompt, respectively. This recurrence mechanism enables RECURRENTGPT to generate texts of arbitrary length without forgetting. Since human users can easily observe and edit the natural language memories, RECURRENTGPT is interpretable and enables interactive generation of long text. RECURRENTGPT is an initial step towards next-generation computer-assisted writing systems beyond local editing suggestions. In addition to producing AI-generated content (AIGC), we also demonstrate the possibility of using RECURRENTGPT as an interactive fiction that directly interacts with consumers. We call this usage of generative models by "**AI a**s **C**ontents" (**AIAC**), which we believe is the next form of conventional AIGC. We further demonstrate the possibility of using RECURRENTGPT to create personalized interactive fiction that directly interacts with readers instead of interacting with writers. More broadly, RECURRENTGPT demonstrates the utility of borrowing ideas from popular model designs in cognitive science and deep learning for prompting LLMs. Our code is available at `https://github.com/aiwaves-cn/RecurrentGPT` and an online demo is available at `https://www.aiwaves.org/recurrentgpt`.

1 Introduction

Large Language Models (LLMs) [1–5] such as ChatGPT have proven to be highly effective tools for assisting with various routine writing tasks, including emails and blog posts. Nevertheless, due to the fixed-size context design inherent in the Transformer [6] architecture, it is unfeasible to generate long texts (e.g., novels) solely by prompting LLMs. In contrast, recurrent neural networks (RNNs) [7, 8], in theory, possess the capacity to generate sequences of arbitrary length, thanks to their recurrence mechanism: RNNs maintain a hidden state that undergoes updates at each time step, employing the current time step's output as the input for the subsequent time step. In practice, however, RNNs suffer from the problem of vanishing and exploding gradients and are hard to scale up.

*Equal Contribution

Preprint. Work In Progress.

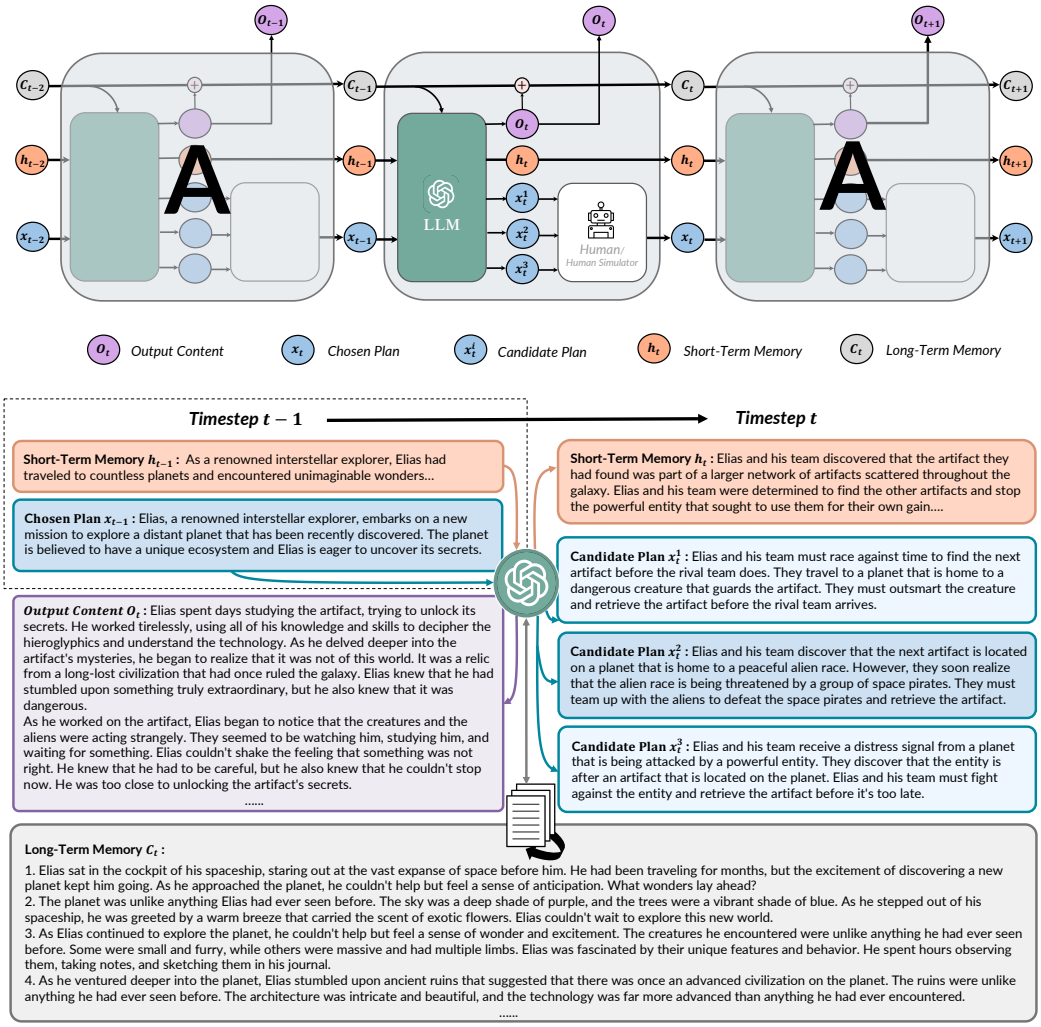

Figure 1: Illustration of the RECURRENTGPT framework. RECURRENTGPT enables recurrent prompting with LLMs by simulating an RNN using natural language building blocks and defines the recurrent computation graph with prompts.

To this end, a number of works [9–11] attempt to equip Transformers with an RNN-like recurrence mechanism. While achieving promising results on long text modeling and generation, these recurrence-augmented Transformers require substantial architectural modifications that have not been proven to scale well. The majority of current LLMs continue to employ the original Transformer architecture with minimal alterations.

In this paper, we introduce RECURRENTGPT, a language-based simulacrum of the recurrence mechanism in RNNs. As illustrated in Figure 1, RECURRENTGPT replaces the vectorized elements (i.e., cell state, hidden state, input, and output) in a Long-short Term Memory RNN (LSTM) [8] with natural language (i.e., paragraphs of texts), and simulates the recurrence mechanism with prompt engineering. At each timestep t, RECURRENTGPT receives a paragraph of text and a brief plan of the next paragraph, which are both generated in step $t-1$. It then attends to the long-term memory, which contains the summaries of all previously generated paragraphs and can be stored on hard drives, and relevant paragraphs can be retrieved with semantic search. RECURRENTGPT also maintains a short-term memory that summarizes key information within recent timesteps in natural language and is updated at each time step. RECURRENTGPT combines all aforementioned inputs in a prompt and asks the backbone LLM to generate a new paragraph, a short plan for the next paragraph, and updates the long-short term memory by rewriting the short-term memory and appending the summary of the output paragraph to the long-term memory. These components are then re-used in the next

time step, resulting in a recurrence mechanism for the generation process. With the language-based recurrence mechanism, RECURRENTGPT alleviates the need for any architectural modification and can be integrated into any powerful LLM, making it capable of generating arbitrarily long text beyond the fixed-size context window.

In addition to surpassing the fixed-size context limitation, RECURRENTGPT enhances the interpretability of the recurrence mechanism in comparison to the vector-based recurrence mechanism employed in RNNs. This improvement stems from the ability to observe the specific segments of long-term memory that are attended to, as well as the manner in which short-term memory is updated, through a simple examination. More importantly, employing natural language as building blocks enables human engagement with RECURRENTGPT, allowing for the human manipulation of its memories and plans for future generations. Human interaction also prevents RECURRENTGPT from deviating from desired behavior, a challenge commonly encountered with recent autonomous GPT-based agents such as AutoGPT[2]. Given that current state-of-the-art computer-assisted writing systems [12, 13] primarily focus on localized editing suggestions and treat LLMs as black-boxes, we believe RECURRENTGPT represents a step towards next-generation computer-assisted writing systems for interactive long text generation that also offer interpretability.

We then extend the utilization of RECURRENTGPT beyond its role as a tool for producing AI-generated content (AIGC) by exploring its potential for direct interaction with consumers, rather than solely with content creators. Specifically, we convert RECURRENTGPT to a personalized interactive fiction wherein it generates multiple prospective plans for the subsequent actions, allowing players to choose and explore the one that captures their interest. Moreover, in addition to selecting from model-generated plans, players possess the capability to devise their own plans. Such a capacity is unattainable within conventional interactive fictions, as the narratives and options are conventionally predetermined. We denote this new paradigm as "AI As Content", signifying the utilization of generative AI as a medium that actively interacts with consumers, instead of being confined to the role of a mere tool for content creators. Through RECURRENTGPT, we perceive a preliminary stride towards a future where AI models will eventually become collaborative partners in our creative endeavors.

In our experiments, we build RECURRENTGPT upon ChatGPT and find that exhibits the capability to autonomously generate remarkably extensive texts, spanning thousands of tokens, while maintaining both coherency and engagement. In stark contrast, vanilla ChatGPT is constrained to generating a few hundred of tokens before encountering issues such as repetitive content or a decline in coherence.Moreover, RECURRENTGPT can help human writers produce arbitrarily long text with ease, reducing much of the human efforts required for writing long creative texts such as novels. The contributions of this paper can be summarized as follows:

- We propose RECURRENTGPT, a language-based simulacrum of the recurrence mechanism in RNNs that mitigates the fixed-size context limitation of LLMs such as ChatGPT.
- We show that RECURRENTGPT can generate very long texts either on its own or serve as an interactive writing assistant, helping human writers write arbitrarily long texts.
- We introduce a new use case of generative AI that uses generative models to directly interact with consumers of text, as opposed to the conventional practice that uses them as tools for content creation, by using RECURRENTGPT as a personalized interactive fiction for content curation.

Furthermore, it is important to underscore that RECURRENTGPT illustrates the possibility of drawing inspiration from well-established model designs in the fields of cognitive science and deep learning, with the aim of generating long form text via prompting of LLMs.

2 RECURRENTGPT

We describe RECURRENTGPT in detail in this section. RECURRENTGPT is a natural language-based counterpart of the recurrence mechanism in RNNs. RECURRENTGPT simulates an LSTM by (1) modeling all vector-based components in an LSTM, including input vectors x_t, output vectors y_t, hidden states h_t, and cell states c_t, with natural language; (2) modeling the recurrent computation

[2] `https://github.com/Significant-Gravitas/Auto-GPT`

graph in an LSTM with natural language prompts, and (3) replacing the trainable parameters in RNNs by a frozen LLM. In theory, the backbone of RECURRENTGPT can be any LLM or text-to-text model, we opt for ChatGPT because of its capability and popularity.

Formally, we define RECURRENTGPT as a computational function parametrized by an LLM with parameter θ and a prompt template $\mathcal{P}$. Recall that the recurrent computation graph of an LSTM can be summarized as:

$$o_{t+1}, h_{t+1}, c_{t+1} = \text{LSTM}(x_{t+1}, h_t, c_t, \theta) \tag{1}$$

where θ denotes the model parameters, x_{t+1} equals to o_t, and h_t, c_t are the long/short-term memories at timestep t, respectively.

By analogy, the recurrence mechanism in our model can be expressed by:

$$o_{t+1}, x_{t+1}, h_{t+1}, c_{t+1} = \text{RECURRENTGPT}(o_t, x_t, h_t, c_t, \theta, \mathcal{P}) \tag{2}$$

where o_t, x_t, h_t, and c_t denote the natural language-based building blocks including content, plan, short-term memory, and long-term memory, at time step t, respectively. Here x_{t+1} does not equal o_t and is instead separately generated, which is different from conventional RNNs. We first describe each building block in RECURRENTGPT and then present how our prompt $\mathcal{P}$ enables RECURRENTGPT to recurrently generate arbitrarily long texts.

2.1 Language-based Building Blocks

Input/Output The input and output of RECURRENTGPT at each timestep include a paragraph of text that gets appended to the final text produced and an outline for the next paragraph to be generated. We refer to these two as the "content" and "plan", respectively. As illustrated in Figure 1, contents typically consist of 200-400 words and should be mostly ready for reading. Whereas plans are outlines for the next content and typically consist of 3-5 sentences. At each timestep, the content and plan generated in the previous timestep are used as input to RECURRENTGPT, allowing recurrent computation. RECURRENTGPT is designed to produce plans in addition to contents as allowing users to read and edit plans increases interpretability and facilitates human-computer interaction.

Long-Short Term Memory Similar to an LSTM, RECURRENTGPT maintains long-short term memory across timesteps. As illustrated in Figure 1, long-term memory summarizes all previously generated contents to minimize information lost when generating long texts. Since the generated content can be arbitrarily long and cannot fit in the context size of LLMs, we implement the long-term memory in RECURRENTGPT with a VectorDB approach by embedding the content generated in each timestep with sentence-transformers [14]. This approach enables RECURRENTGPT to store even longer memory compared to previous memory-based Transformers [9, 11] as it can store memory in disk space instead of GPU memory. This can be important in several use cases where the users may not have high-end GPUs in their devices.

Short-term memory, on the other hand, is a short paragraph of texts summarizing key information across recent timesteps. The length of the short-term memory is controlled to 10-20 sentences so that it can fit into the prompt and can be updated by the LLM backbone. By combining long-short term memory, RECURRENTGPT can maintain coherence with recently generated content and also recall key information that was generated long before. This is impossible with vanilla LLMs because they can only take a few previously generated texts in the input.

RECURRENTGPT can be initialized using a simple prompt that instructs the LLM to generate the aforementioned components with texts specifying the topic of the novel and other background information. When using RECURRENTGPT to continue writing a novel, users can write down (or prompt ChatGPT to generate) a short-term memory and an initial plan.

2.2 Language-based Recurrent Computation

While RNNs achieve recurrent computation by implementing a feedback loop in the computation graph, RECURRENTGPT relies on prompt engineering to simulate the recurrent computation scheme. As illustrated in Figure 1, RECURRENTGPT simulates the computation graph in RNNs with a prompt template, which is presented in Figure 1 in the Appendix, and some simple Python code[3].

[3]We present the prompt in Appendix A due to space constraints.

At each timestep, RECURRENTGPT constructs the input prompts by filling the prompt template with input content/plan and its internal long-short term memory. In particular, since the long-term memory cannot fit into the context size, we use the input plan as the query to perform a semantic search over the VectorDB-based long-term memory and fit a few most relevant contents into the prompt. The prompt then instructs the LLM backbone to generate new contents, plans, and updated short-term memory. As illustrated in Figure 1 in the Appendix, our prompt encourages the LLM to update the short-term memory by discarding information that is no longer relevant and adding useful new information while maintaining its length within a range so that it can always fit in the context size. It is noteworthy that we prompt the LLM to generate multiple (e.g., 3 in our experiments) plans. This improves the diversity of outputs and makes human-computer interaction more friendly by allowing human users to select the most suitable plan. We also give users the option to write plans on their own if none of the generated plans is desirable. To make RECURRENTGPT capable of generating long texts autonomously without human intervention, we add a prompt-based human simulator to select a good plan and revise it for the next timestep.

2.3 Interactive Long Text Generation with RECURRENTGPT

While RECURRENTGPT can generate long texts on its own with the recurrence mechanism, its language-based computation scheme offers unique interpretability and interactivity. Compared to conventional computer-assisted writing systems that use language models as black boxes and only give next phrase/sentence suggestions, RECURRENTGPT enjoys the following advantages:

- It is more efficient at reducing human labor because it makes paragraph/chapter-level progresses instead of local writing suggestions.
- It is interpretable because users can directly observe its language-based internal states.
- It is interactive because humans can edit their building blocks with natural language.
- It is customizable because users can easily modify the prompts to customize the model according to their own interests (e.g., the style of output texts, how much progress to make for each timestep, etc.)

In addition, human interaction can also help correct accidental mistakes made by RECURRENTGPT when autonomously generating long texts and prevent error propagation, which is a major bottleneck for long text generation.

3 Experiments

3.1 Experimental Settings

Tasks We test the empirical effectiveness of RECURRENTGPT in this section. In particular, we evaluate RECURRENTGPT in three different settings including:

- Autonomously generating long texts without human interaction.
- Collaboratively generating long texts with a human writer
- Directly interacting with text consumers as interactive fictions.

In each of these tasks, we test with a diverse set of genres of novels including science fiction, romance, fantasy, horror, mystery, and thriller novels. To test the effectiveness of RECURRENTGPT for texts of different length, we generate novels of medium length ($\sim$ 3000 words) for horror, mystery, and thriller, and generate longer novels ($\sim$ 6000 words) for sci-fi, romance, and fantasy.

Baselines Although RECURRENTGPT is the first work on using LLMs to generate arbitrarily long texts, we can still compare it against some reasonable baselines and ablated variants, as listed below:

- **Rolling-ChatGPT**, a simple baseline that prompts ChatGPT to start writing a novel given a genre of literature and some outlines or background settings, and then iteratively prompts ChatGPT to continue writing after reaching the context length limit. This baseline is roughly equivalent to using a sliding context window trick for generating long texts with Transformers.

- **RE**^{3} [15] is a hierarchical long story generation baseline that first prompts an LLM to generate an outline for the story and then generates the story following the outline with some re-ranking and re-writing pipelines. We re-implement it with ChatGPT to ensure a fair comparison.
- **DOC** [16] is the state-of-the-art long story generation baseline that improves **RE**^{3} with outline control. We re-implement DOC by replacing OPT-175B [17] with ChatGPT and removing the detailed controller, which is impossible to use because we do not have access to ChatGPT weights. In general, we find that our re-implementation results in slightly better quality because of the improvement on the backbone LLM.

It's noteworthy that in principle, both the baselines can not generate arbitrarily long texts while remaining coherent. This is because the **Rolling-ChatGPT** baseline forgets previously generated contents very quickly. On the other hand, **RE**^{3} and **DOC** fixes the outline in the first stage, which limits the overall length of the story to be generated.

Table 1: Pair-wise comparison of RECURRENTGPT with baselines for 20 novels of different genres. Results in different comparisons are not comparable with each other. Bold indicates significance with $p < 0.05$.

Novel genres	Sci-fi		Romance		Fantasy	
$\sim$ 6000 words	Interesting ↑	Coherent ↑	Interesting ↑	Coherent ↑	Interesting ↑	Coherent ↑
RECURRENTGPT	**94.7**	**86.5**	**91.4**	**84.8**	**95.9**	**85.1**
Rolling-ChatGPT	7.8	14.3	9.0	18.2	6.5	13.7
RECURRENTGPT	**68.3**	**65.7**	**71.4**	**69.2**	**63.8**	**62.0**
RE^3	31.9	28.5	28.1	25.3	35.1	33.8
RECURRENTGPT	**66.1**	**59.3**	**77.2**	**63.4**	**61.0**	**56.5**
DOC	30.7	38.1	25.3	29.8	31.2	40.3
Novel genres	Horror		Mystery		Thriller	
$\sim$ 3000 words	Interesting ↑	Coherent ↑	Interesting ↑	Coherent ↑	Interesting ↑	Coherent ↑
RECURRENTGPT	**88.3**	**84.9**	**87.1**	**82.0**	**91.5**	**82.7**
Rolling-ChatGPT	13.5	17.1	14.5	20.1	11.9	17.7
RECURRENTGPT	**64.1**	**64.5**	**66.8**	**63.2**	**61.0**	**61.4**
RE^3	34.6	30.2	27.9	28.8	38.3	37.9
RECURRENTGPT	**65.8**	**60.7**	**72.1**	**66.8**	**60.2**	**58.1**
DOC	29.1	39.7	27.2	25.6	33.8	37.0

Evaluation Metrics For evaluation, we follow Yang et al. [15] and conduct a human evaluation by comparing RECURRENTGPT with the baselines according to two dimensions:

- **Interesting**: How interesting are the generated novels for common readers?
- **Coherent:** How well are the paragraphs organized and connected with each other?

We omit the "quality" or "humanlike" metrics following Yang et al. [16] since all baselines are based on ChatGPT which can produce high-quality texts most of the time. We evaluate the compared models by pairwise comparison. Specifically, we give two novels (A and B, with random order) generated by different compared methods to human annotators with good English proficiency and instruct them to label whether novel A or novel B is better, or they are indistinguishable, in terms of interestingness and coherence. Following the human evaluation settings in Yang et al. [16], we sample 20 generated novels for each genre and assign 3 annotators for each novel.

3.2 Results

As shown in Table 1, we find that RECURRENTGPT is favored by human readers for both interestingness and coherence with a relatively large margin compared to both the rolling-window baseline and prior state-of-the-arts like RE^3 and DOC. This confirms our intuition that recurrent computation is important for long text generation. The gap is larger for longer novels, which confirms the advantage of

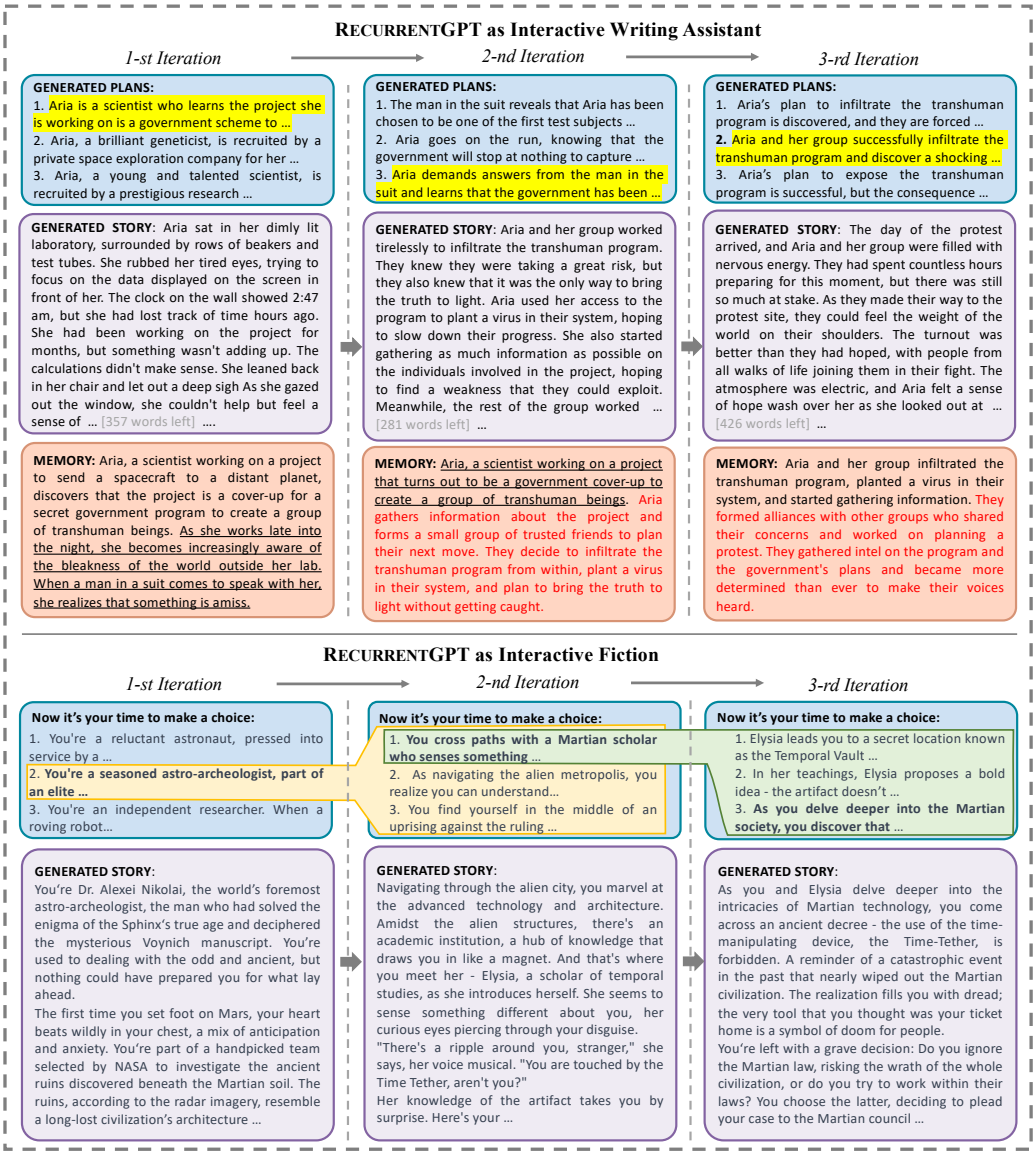

Figure 2: Qualitative analysis of using RECURRENTGPT as an interactive writing assistant and an interactive fiction. Highlighted plans or choices are that selected by human users.

RECURRENTGPT on generating very long texts. Finally, human annotators prefer RECURRENTGPT in all novel genres. This confirms its robustness on different types of long texts.

To better understand the effectiveness of RECURRENTGPT, we also conduct an ablation study by comparing RECURRENTGPT with with ablated variants without either short-term or long-term memory, and the variant that uses GPT-4 as the backbone model. The results are shown in Table 2. We can see that long/short-term memory mainly contributes to the coherence of generated texts, which correlates well with our intuition. RECURRENTGPT with GPT-4 as the backbone LLM is drastically favored compared to its counterpart using ChatGPT/GPT-3.5-turbo. This confirms the potential of RECURRENTGPT when equipped with more powerful LLMs. We present a few sample novels generated by RECURRENTGPT in the Appendix for qualitative evaluation.

3.3 RECURRENTGPT as Interactive Writing Assistant

We then test the usefulness of RECURRENTGPT as an interactive writing assistant from a human-AI interaction perspective. As illustrated in Figure 2, a human writer starts by choosing the topic he/she

Table 2: Pair-wise comparison of RECURRENTGPT with ablated variants and the variant that uses GPT-4 as the backbone model. We sample 20 novels of different genres for comparison. Results in different comparisons are not comparable with each other. Bold indicates significance with $p < 0.05$.

Novel genres	Sci-Fi		Fantasy	
$\sim$ 6000 words	Interesting ↑	Coherent ↑	Interesting ↑	Coherent ↑
RECURRENTGPT	58.9	**65.1**	55.3	**64.1**
w/o Short term memory	44.2	31.0	47.7	33.5
RECURRENTGPT	51.4	**71.3**	57.5	**68.9**
w/o Long term memory	40.0	27.8	46.2	38.7
RECURRENTGPT	21.3	28.1	27.1	24.8
w/ GPT-4	**73.4**	**64.9**	**71.7**	**70.5**

wants to write and writes a short paragraph describing the background and the outline of the book. Then RECURRENTGPT automatically generates the first paragraphs and provides a few possible options for the writer to continue the story. The writer may select one from them and edit it if needed. He or she can also write a short plan for the next few paragraphs by him/herself if generated plans are all inappropriate, which makes human-AI co-writing process more flexible. We show a Gradio[4]-based interface that allows human writers to write different genres of novels by interacting with RECURRENTGPT in Appendix B.

According to a small-scale human user study, RECURRENTGPT significantly improves the productivity of human writers[5], and the improvements mainly come from: (1) reducing the time for typing long texts by writing or choosing short plans and letting RECURRENTGPT generate the actual texts; and (2) reducing the time for designing less important plots by selecting plans from RECURRENTGPT generated ones, according to user feedback. Moreover, users feel that RECURRENTGPT is more interpretable and controllable compared to conventional AI writing assistants that act as black-boxes since the language-based components in RECURRENTGPT are transparent and editable for users. Finally, compared to the previous methods that hierarchically generate long texts such as DOC and RE^3, human users prefer our system since iteratively and interactively writing long texts is more flexible and controllable. Finally, our system is very different from most existing AI writing assistants since they focus on providing local writing suggestions within phrases or a few sentences, whereas RECURRENTGPT can generate a few paragraphs at a time.

3.4 RECURRENTGPT as Interactive Fiction

We also test the possibility of using RECURRENTGPT as personalized interactive fiction. This use case is very similar to RECURRENTGPT as AI writing assistants. The main differences are two-fold as illustrated in Figure 2: (1) the shift from the third-person perspective to the first-person perspective, which aims to foster a sense of immersion for human players, and (2) making RECURRENTGPT generate plans that involve important choices for the main character as opposed to general plans for the next paragraphs. The adaptation can be easily implemented by slightly modifying the prompt.

Our user study shows that RECURRENTGPT can interact with human players and directly provide content of good quality for human consumers. Human players also find the possibility of writing free-form texts as their actions in interactive fiction largely improve their interestingness. This confirms the potential of directly using generative AI as content, instead of using them as tools to produce content. However, we also find that RECURRENTGPT sometimes produces less consistent content and low-quality options that are not very relevant or reasonable. We believe this can be improved by using a more powerful LLM backbone, fine-tuning the LLM backbone with supervised fine-tuning or reinforcement learning from human feedback, or designing better prompts. We leave this for future work.

[4]https://gradio.app/

[5]We will conduct a larger-scale user study and present the details and results in the revised version.

4 Related Works

4.1 Transformers Beyond Fixed-size Context

One major limitation of Transformers is that the context size is fixed, which hinders their ability on processing and producing long texts. Previous work attempts to solve this issue from two different ways: designing efficient attention mechanisms to train and use Transformers with larger context windows [18–21], and adding memory mechanisms to the computational graph in a Transformer to allow it to process information from multiple context windows [9, 22, 23, 11]. While these methods enable Transformers to process very long texts, they all require substantial architectural changes to the original Transformer architecture. Therefore, these approaches can not be integrated into powerful pre-trained LLMs such as ChatGPT and LLAMA, which substantially limits their usefulness. Recently, Press et al. [24] introduces ALiBi, which adds linear bias to attention to allow input length extrapolation. However, this method mainly supports longer inputs instead of longer outputs. In addition, it requires access to the model parameters and inference codes, which is often not possible since many state-of-the-art LLMs such as ChatGPT, GPT-4, and PaLM, are closed-sourced.

4.2 Long Text Generation

In addition to architectural modifications, a number of works investigate long text generation in a hierarchical manner. Fan et al. [25] first propose to generate a story by first generating a short summary of it and then improve this method by adding an intermediate step of generating an outline which is the predicate-argument structure of the story [26]. Tan et al. [27] and Sun et al. [28] further improve this kind of hierarchical long text generation method. Yao et al. [29] also propose to first generate a storyline and then complete the story. This line of research is further improved by RE^3[15] and its variant DOC[16], which proposed to recursively prompt LLMs for long story generation in a plan-and-write fashion. However, the plots and length of their final stories are still constrained by the pre-determined plans. In contrast, RECURRENTGPT overcomes the above limitations via recurrent generation, which enables effective human-LM collaboration and improves the flexibility and controllability for long text generation.

4.3 AI-Assisted Writing Systems

AI writing assistants have been adopted in a variety of applications, including story completion[12], essay writing [30], and poem generation [31]. Existing systems can be broadly classified into *interactive* generation and *automatic* generation. Interactive systems [32–34] are mainly designed to provide local suggestions or revisions at the phrase or sentence level. As a result, they are less able to ease the creative burden for human writers. On the other hand, automatic generation [26, 35, 36] aims to write full texts based on given prompts or topics via the sequence-to-sequence framework. Although advances in LLMs have demonstrated impressive potential for these systems, the lack of transparency, controllability, and sense of collaboration could harm user experience regarding writers' perceived ownership [12, 37]. Besides, most of them are limited by providing local editing suggestions ranging from several phrases to a few sentences [38, 39], partly due to the length limitation of NLG models and partly due to the challenge of maintaining long-range coherence.

5 Limitations

One limitation of this work is that while RECURRENTGPT can generate arbitrarily long texts, we only evaluate it on settings where the generated texts are at most around 5000 words. This is because both qualitative and quantitive evaluations of very long texts are prohibitively hard. Another limitation is that RECURRENTGPT only works with backbone LLMs that are powerful enough such as ChatGPT and GPT-4. We believe this issue can be alleviated when more powerful smaller LLMs are developed. Finally, our user study for evaluating RECURRENTGPT as an AI writing assistant and as interactive fiction is limited by small-scale studies. We will add larger and more throughout the user study in the revised version. As for the social impact, RECURRENTGPT can improve the quality of AI-generated long texts and increase the productivity of human writers. However, it can also be misused to generate garbage or harmful content that leads to negative social impact. However, this is a known limitation of generative AI and we will make our best effort to promote responsible usage of generative AI.

6 Conclusions

We present RECURRENTGPT, a language-based simulacra of the recurrence mechanism in RNNs that uses language-based components and defines a recurrent computation graph via prompt engineering. RECURRENTGPT enbale LLMs to generate arbitrarily long texts either autonomously or by interacting with human writters. Its language-based components improves its interpretability and controllability and the prompt-based computation graph makes it easily customizable. User study on using RECURRENTGPT as AI writing assistants and text-based games demonstrates its potential as an initial step towards next-generation AI writing assistant beyond local writing suggestions and directly using generative AI as contents that are consumerable via interaction. Finally, our work also demonstates the possibility of borrowing ideas from popular model designs in cognitive science and deep learning literature for long form text generation using LLMs.

References

[1] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. 2018.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[3] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf.

[4] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Gray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=TG8KACxEON.

[5] OpenAI. Gpt-4 technical report, 2023.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.

[7] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. ISSN 0364-0213. doi: https://doi.org/10.1016/0364-0213(90)90002-E. URL https://www.sciencedirect.com/science/article/pii/036402139090002E.

[8] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[9] Zihang Dai*, Zhilin Yang*, Yiming Yang, William W. Cohen, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. Transformer-XL: Language modeling with longer-term dependency, 2019. URL https://openreview.net/forum?id=HJePno0cYm.

[10] Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=SylKikSYDH.

[11] Aydar Bulatov, Yuri Kuratov, and Mikhail Burtsev. Recurrent memory transformer. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022. URL https://openreview.net/forum?id=Uynr3iPhksa.

[12] Mina Lee, Percy Liang, and Qian Yang. Coauthor: Designing a human-ai collaborative writing dataset for exploring language model capabilities. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, CHI '22, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450391573. doi: 10.1145/3491102.3502030. URL https://doi.org/10.1145/3491102.3502030.

[13] Hai Dang, Sven Goller, Florian Lehmann, and Daniel Buschek. Choice over control: How users write with large language models using diegetic and non-diegetic prompting. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9781450394215. doi: 10.1145/3544548.3580969. URL https://doi.org/10.1145/3544548.3580969.

[14] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 11 2019. URL https://arxiv.org/abs/1908.10084.

[15] Kevin Yang, Yuandong Tian, Nanyun Peng, and Dan Klein. Re3: Generating longer stories with recursive reprompting and revision. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 4393–4479, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.emnlp-main.296.

[16] Kevin Yang, Dan Klein, Nanyun Peng, and Yuandong Tian. Doc: Improving long story coherence with detailed outline control, 2022.

[17] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[18] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.

[19] Nikita Kitaev, Lukasz Kaiser, and Anselm Levskaya. Reformer: The efficient transformer. In *ICLR*. OpenReview.net, 2020.

[20] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019.

[21] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *NeurIPS*, 2020.

[22] Zhiwei Wang, Yao Ma, Zitao Liu, and Jiliang Tang. R-transformer: Recurrent neural network enhanced transformer, 2019.

[23] Peng Cui and Le Hu. Sliding selector network with dynamic memory for extractive summarization of long documents. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5881–5891, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.470. URL https://aclanthology.org/2021.naacl-main.470.

[24] Ofir Press, Noah A. Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. In *ICLR*. OpenReview.net, 2022.

[25] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 889–898, 2018.

[26] Angela Fan, Mike Lewis, and Yann Dauphin. Strategies for structuring story generation. *arXiv preprint arXiv:1902.01109*, 2019.

[27] Bowen Tan, Zichao Yang, Maruan Al-Shedivat, Eric Xing, and Zhiting Hu. Progressive generation of long text with pretrained language models. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4313–4324, Online, June 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.naacl-main.341. URL `https://aclanthology.org/2021.naacl-main.341`.

[28] Xiaofei Sun, Zijun Sun, Yuxian Meng, Jiwei Li, and Chun Fan. Summarize, outline, and elaborate: Long-text generation via hierarchical supervision from extractive summaries. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 6392–6402, Gyeongju, Republic of Korea, October 2022. International Committee on Computational Linguistics. URL `https://aclanthology.org/2022.coling-1.556`.

[29] Lili Yao, Nanyun Peng, Ralph M. Weischedel, Kevin Knight, Dongyan Zhao, and Rui Yan. Plan-and-write: Towards better automatic storytelling. In *AAAI*, pages 7378–7385. AAAI Press, 2019.

[30] Yuanchao Liu, Bo Pang, and Bingquan Liu. Neural-based Chinese idiom recommendation for enhancing elegance in essay writing. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5522–5526, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1552. URL `https://aclanthology.org/P19-1552`.

[31] Marjan Ghazvininejad, Xing Shi, Jay Priyadarshi, and Kevin Knight. Hafez: an interactive poetry generation system. In *Proceedings of ACL 2017, System Demonstrations*, pages 43–48, Vancouver, Canada, July 2017. Association for Computational Linguistics. URL `https://aclanthology.org/P17-4008`.

[32] Andy Coenen, Luke Davis, Daphne Ippolito, Emily Reif, and Ann Yuan. Wordcraft: a human-ai collaborative editor for story writing. *arXiv preprint arXiv:2107.07430*, 2021.

[33] John Joon Young Chung, Wooseok Kim, Kang Min Yoo, Hwaran Lee, Eytan Adar, and Minsuk Chang. Talebrush: sketching stories with generative pretrained language models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems*, pages 1–19, 2022.

[34] Seraphina Goldfarb-Tarrant, Haining Feng, and Nanyun Peng. Plan, write, and revise: an interactive system for open-domain story generation. *arXiv preprint arXiv:1904.02357*, 2019.

[35] Yufei Tian and Nanyun Peng. Zero-shot sonnet generation with discourse-level planning and aesthetics features. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3587–3597, Seattle, United States, July 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.naacl-main.262. URL `https://aclanthology.org/2022.naacl-main.262`.

[36] Boyang Li, Stephen Lee-Urban, George Johnston, and Mark Riedl. Story generation with crowdsourced plot graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 27, pages 598–604, 2013.

[37] Jeremy Birnholtz, Stephanie Steinhardt, and Antonella Pavese. Write here, write now! an experimental study of group maintenance in collaborative writing. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pages 961–970, 2013.

[38] Rujun Han, Hong Chen, Yufei Tian, and Nanyun Peng. Go back in time: Generating flashbacks in stories with event temporal prompts. *arXiv preprint arXiv:2205.01898*, 2022.

[39] Lili Yao, Nanyun Peng, Ralph Weischedel, Kevin Knight, Dongyan Zhao, and Rui Yan. Plan-and-write: Towards better automatic storytelling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 7378–7385, 2019.

A Prompts

```
I need you to help me write a novel. Now I give you a memory (a brief summary) of 400 words, you should use it to store the key content of what has
been written so that you can keep track of very long context. For each time, I will give you your current memory (a brief summary of previous
stories. You should use it to store the key content of what has been written so that you can keep track of very long context), the previously
written paragraph, and instructions on what to write in the next paragraph. I need you to write:
1. Output Paragraph: the next paragraph of the novel. The output paragraph should contain around 20 sentences and should follow the input
instructions.
2. Output Memory: The updated memory. You should first explain which sentences in the input memory are no longer necessary and why, and then
explain what needs to be added into the memory and why. After that you should write the updated memory. The updated memory should be similar to the
input memory except the parts you previously thought that should be deleted or added. The updated memory should only store key information. The
updated memory should never exceed 20 sentences!
3. Output Instruction:  instructions of what to write next (after what you have written). You should output 3 different instructions, each is a
possible interesting continuation of the story. Each output instruction should contain around 5 sentences
    Here are the inputs:

    Input Memory:
    {short_memory}

    Input Paragraph:
    {input_paragraph}

    Input Instruction:
    {input_instruction}

    Input Related Paragraphs:
    {input_long_term_memory}

    Now start writing, organize your output by strictly following the output format as below:
    Output Paragraph:
    <string of output paragraph>, around 20 sentences.

    Output Memory:
    Rational: <string that explain how to update the memory>;
    Updated Memory: <string of updated memory>, around 10 to 20 sentences

    Output Instruction:
    Instruction 1: <content for instruction 1>, around 5 sentences
    Instruction 2: <content for instruction 2>, around 5 sentences
    Instruction 3: <content for instruction 3>, around 5 sentences

Very important: The updated memory should only store key information. The updated memory should never contain over 500 words! Finally, remember
that you are writing a novel. Write like a novelist and do not move too fast when writing the output instructions for the next paragraph. Remember
that the chapter will contain over 10 paragraphs and the novel will contain over 100 chapters. And this is just the begining. Just write some
interesting staffs that will happen next. Also, think about what plot can be attractive for common readers when writing output instructions. You
should first explain which sentences in the input memory are no longer necessary and why, and then explain what needs to be added into the memory
and why. After that, you start rewrite the input memory to get the updated memory.
```

Figure 3: The prompts designed for the backbone LLM in the RECURRENTGPT framework that simulates input (plan, instruction), output, short-term memory, and long-term memory, respectively.

B Demo

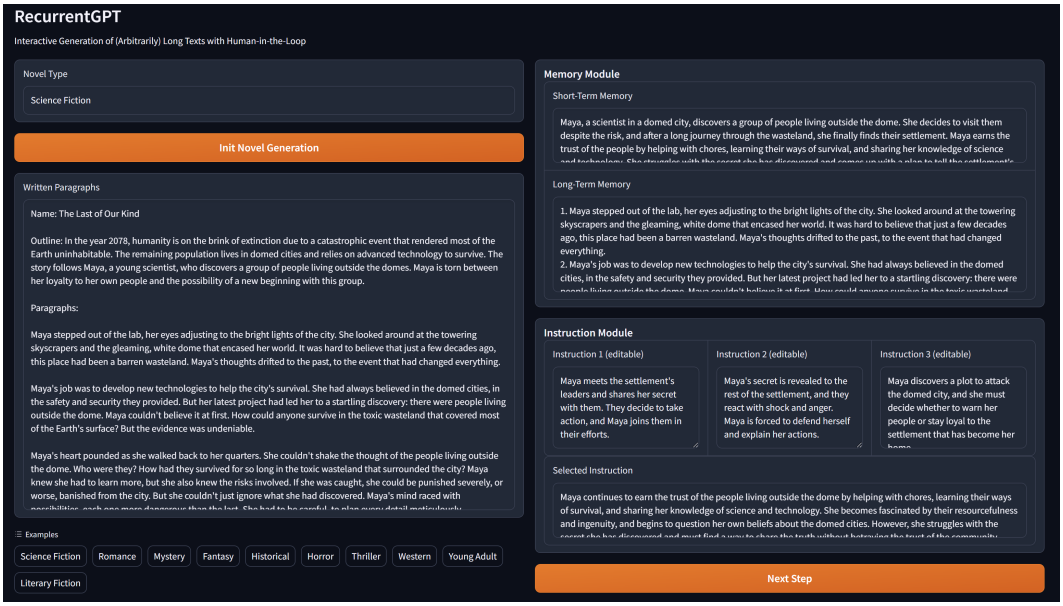

Figure 4: A web demo of RECURRENTGPT.